CLOUD & DEVOPS

Distributed Tracing Visual Guide

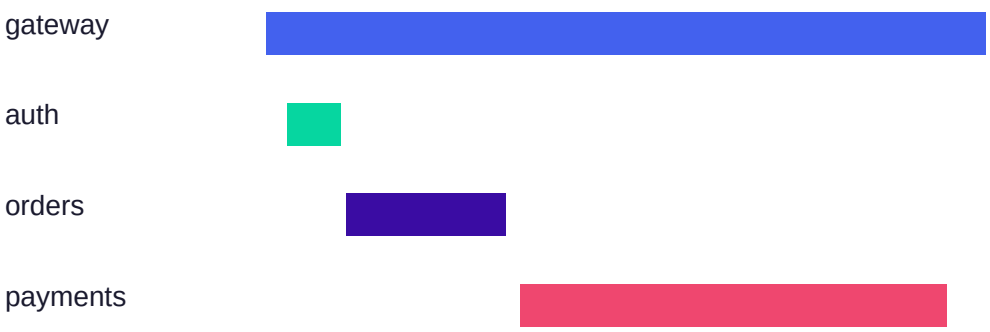
Spans, traces, sampling —
follow one request across fifty services.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Spans & Traces

A trace is a tree of spans — one span per operation.



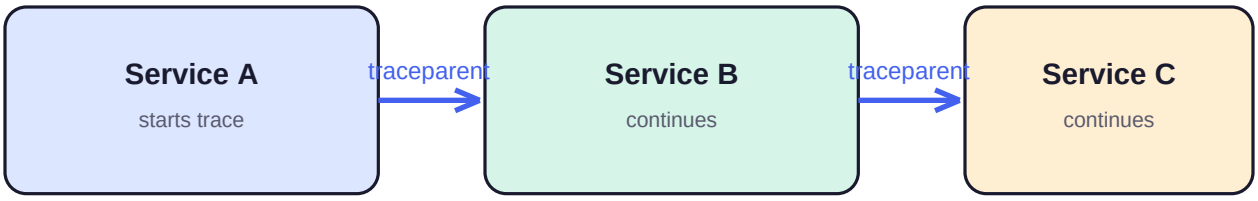
Time →. The payments span dominates: 160ms of a 270ms request. That's your bottleneck.

- Span = one operation: name, start/end, attributes, parent span ID.
- Trace = the full tree, joined by a single trace_id across services.
- The waterfall makes the slow span obvious — no guessing.

STEP 2/6

Context Propagation

The trace_id rides along in headers — every hop continues it.




```
traceparent: 00-<trace_id>-<parent_span_id>-01
```

W3C standard header — libraries propagate it automatically if you use OpenTelemetry.

Break propagation (custom HTTP client without instrumentation) and the trace splits in two — instrument everything.

STEP 3/6

Jaeger / Tempo Architecture

Collectors receive spans; the backend stores and queries them.

OpenTelemetry Collector: vendor-neutral pipeline — receive, process, export to any backend.

- SDKs batch spans and export async — tracing must never block the request path.
- The collector adds environments, filters PII, and fans out to multiple backends.
- Tempo: object-storage backend, cheap at scale — traces as blobs, indexed by ID.

STEP 4/6

Sampling: You Can't Keep Everything

1M req/s x full traces = bankruptcy. Sample smartly.

Head-based
decide at start

- Cheap: drop spans early
- Misses rare errors (1% sample)
- Consistent via trace_id hash

Tail-based
decide at end

- Sees the full trace first
- Keeps ALL errors + slow
- Needs a collector buffer

Tail-based sampling is the gold standard: keep 100% of errors and slow traces, 1% of the boring ones.

STEP 5/6

What to Put in a Span

Attributes turn a timeline into a diagnosis.

http.route

/checkout — not /checkout/42 (cardinality!)

db.statement

the slow query text, sanitized

error

true + exception message on failures

user tier

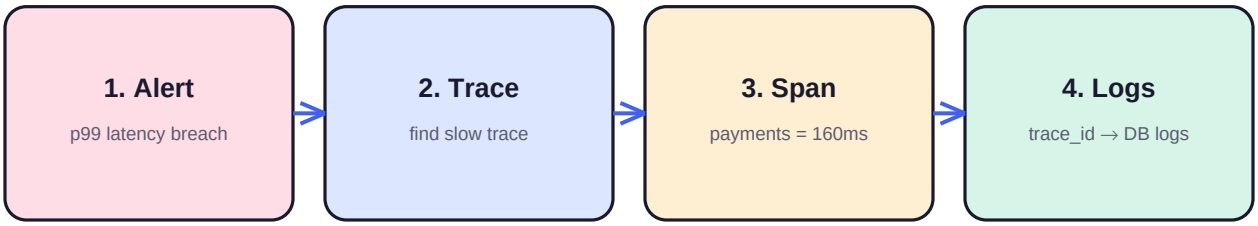
free vs paid — is slowness tiered?

Cardinality rule: attribute VALUES must be bounded. user_id as an attribute = index explosion.

STEP 6/6

Trace-Driven Debugging

The workflow: slow request → trace → slow span → logs.



Exemplars: link Prometheus histograms to example traces — click a slow bucket, see the trace.

STEP ✓/✓

Key Takeaways

Distributed Tracing — Remember

- A trace is a tree of spans; the waterfall exposes the slow operation instantly.
- Propagate trace_id via W3C traceparent headers — instrument every hop.
- OpenTelemetry Collector: vendor-neutral receive → process → export pipeline.
- Tail-based sampling keeps all errors and slow traces, drops the boring 99%.
- Debug flow: alert → trace → slow span → logs via trace_id.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev